

MVLU COLLEGE.

Aim :- A) Implement the Breadth First Search algorithm to solve a given problem.

```
from collections import deque

graph = {
    "Andheri": [("WEH", 2.0), ("Santacruz", 3.0)],
    "WEH": [("Vakola", 2.5)],
    "Vakola": [("Bandra Flyover", 3.0)],
    "Bandra Flyover": [("BKC", 3.6)],
    "Santacruz": [("Kalina", 3.0)],
    "Kalina": [("BKC", 5.1)],
    "BKC": []
}

def bfs(graph, start, goal):
    queue = deque([(start, [start], 0)])
    visited = set()

    while queue:
        node, path, distance = queue.popleft()

        if node == goal:
            return path, distance

        if node not in visited:
            visited.add(node)

            for neighbour, dist in graph[node]:
                queue.append((neighbour, path + [neighbour], distance +
dist))

path, distance = bfs(graph, "Andheri", "BKC")

print("BFS Route:")
print(" -> ".join(path))
print("Total Distance:", distance, "km")
```

MVLU COLLEGE.

The image displays two screenshots of a Jupyter Notebook interface, showing a Breadth-First Search (BFS) implementation in Python. The notebook is titled "BFS_DFS_PRACT01_T100.ipynb".

First Screenshot: The code defines a graph and a BFS function.

```
[1] ✓ 0s from collections import deque

graph = {
    "Andheri": [{"WEH": 2.0}, {"Santacruz": 3.0}],
    "WEH": [{"Vakola": 2.5}],
    "Vakola": [{"Bandra Flyover": 3.0}],
    "Bandra Flyover": [{"BKC": 3.6}],
    "Santacruz": [{"Kalina": 3.0}],
    "Kalina": [{"BKC": 5.1}],
    "BKC": []
}

def bfs(graph, start, goal):
    queue = deque([(start, [start], 0)])
    visited = set()

    while queue:
        node, path, distance = queue.popleft()

        if node == goal:
            return path, distance

        if node not in visited:
            visited.add(node)

            for neighbour, dist in graph[node]:
                queue.append((neighbour, path + [neighbour], distance + dist))

    path, distance = bfs(graph, "Andheri", "BKC")
```

Second Screenshot: The code is executed, and the output is displayed.

```
[2] ✓ 0s "Santacruz": [{"Kalina": 3.0}],
"Kalina": [{"BKC": 5.1}],
"BKC": []
}

def bfs(graph, start, goal):
    queue = deque([(start, [start], 0)])
    visited = set()

    while queue:
        node, path, distance = queue.popleft()

        if node == goal:
            return path, distance

        if node not in visited:
            visited.add(node)

            for neighbour, dist in graph[node]:
                queue.append((neighbour, path + [neighbour], distance + dist))

    path, distance = bfs(graph, "Andheri", "BKC")

print("BFS Route:")
print(" -> ".join(path))
print("Total Distance:", distance, "km")

BFS Route:
Andheri -> Santacruz -> Kalina -> BKC
Total Distance: 11.1 km
```

NANDINI PANDIT T100
Artificial Intelligence.

MVLU COLLEGE.

B) Implement the Iterative Depth First Search algorithm to solve the same problem.

```
graph = {
    "Andheri": [("WEH", 2.0), ("Santacruz", 3.0)],
    "WEH": [("Vakola", 2.5)],
    "Vakola": [("Bandra Flyover", 3.0)],
    "Bandra Flyover": [("BKC", 3.6)],
    "Santacruz": [("Kalina", 3.0)],
    "Kalina": [("BKC", 5.1)],
    "BKC": []
}

def dfs(graph, start, goal):
    stack = [(start, [start], 0)]
    visited = set()

    while stack:
        node, path, distance = stack.pop()

        if node == goal:
            return path, distance

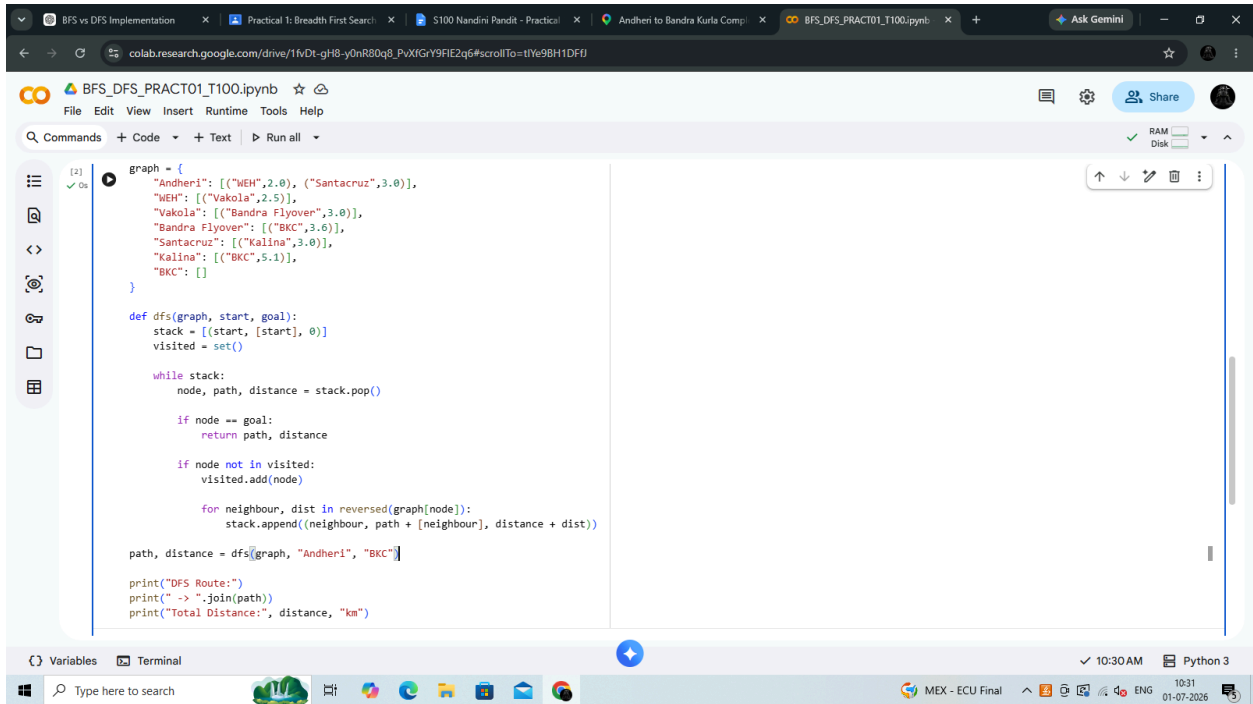
        if node not in visited:
            visited.add(node)

            for neighbour, dist in reversed(graph[node]):
                stack.append((neighbour, path + [neighbour], distance +
dist))

path, distance = dfs(graph, "Andheri", "BKC")

print("DFS Route:")
print(" -> ".join(path))
print("Total Distance:", distance, "km")
```

MVLU COLLEGE.



The screenshot shows a Google Colab notebook titled "BFS_DFS_PRACT01_T100.ipynb". The code defines a graph and a DFS function. The graph has nodes: "Andheri", "MEH", "Vakola", "Bandra Flyover", "Santacruz", "Kalina", and "BKC". The DFS function starts at "Andheri" and aims to reach "BKC".

```
graph = {
    "Andheri": [{"MEH", 2.0}, {"Santacruz", 3.0}],
    "MEH": [{"Vakola", 2.5}],
    "Vakola": [{"Bandra Flyover", 3.0}],
    "Bandra Flyover": [{"BKC", 3.0}],
    "Santacruz": [{"Kalina", 3.0}],
    "Kalina": [{"BKC", 5.1}],
    "BKC": []
}

def dfs(graph, start, goal):
    stack = [(start, [start], 0)]
    visited = set()

    while stack:
        node, path, distance = stack.pop()

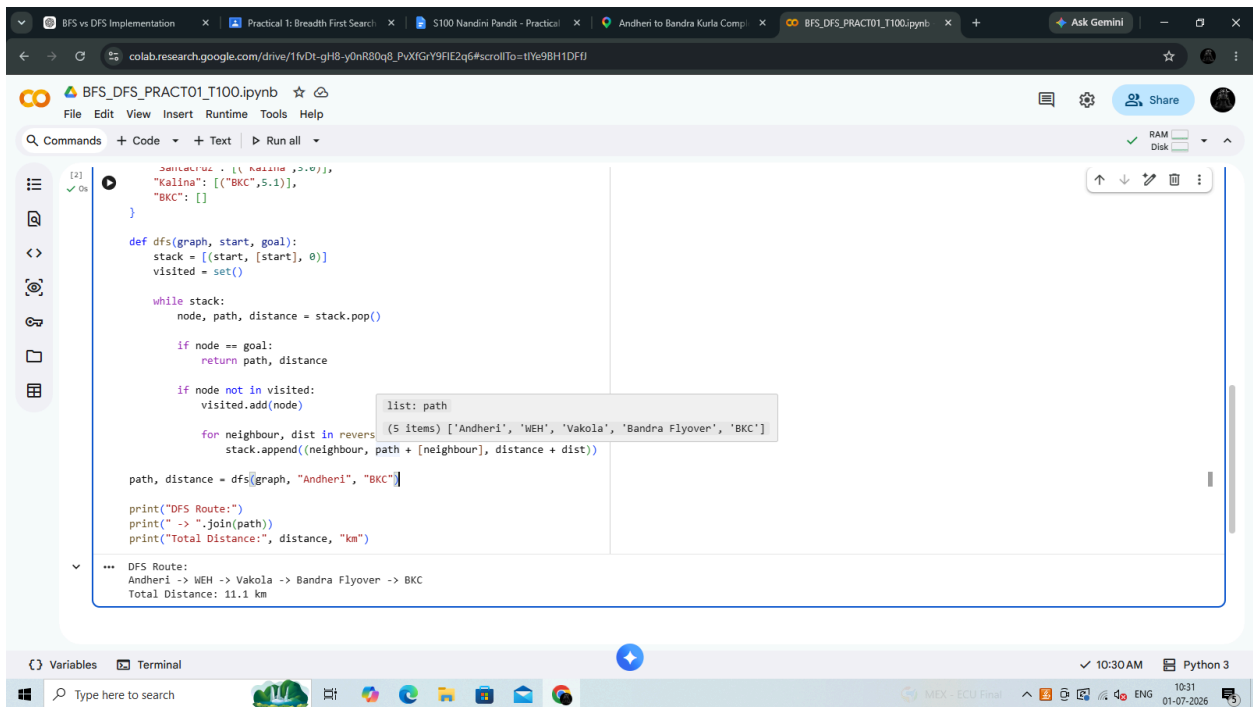
        if node == goal:
            return path, distance

        if node not in visited:
            visited.add(node)

            for neighbour, dist in reversed(graph[node]):
                stack.append((neighbour, path + [neighbour], distance + dist))

    path, distance = dfs(graph, "Andheri", "BKC")

print("DFS Route:")
print(" -> ".join(path))
print("Total Distance:", distance, "km")
```



The screenshot shows the same Google Colab notebook after execution. The output displays the DFS route and total distance. A tooltip shows the path list: ["Andheri", "MEH", "Vakola", "Bandra Flyover", "BKC"].

```
DFS Route:
Andheri -> MEH -> Vakola -> Bandra Flyover -> BKC
Total Distance: 11.1 km
```

MVLU COLLEGE.

C) Compare the performance and efficiency of both algorithms.

```
print("Comparison of BFS and DFS")
print("-" * 40)

print("Vertices (V) = 7")
print("Edges (E) = 7\n")

print("BFS Path : [1, 5, 6, 7]")
print("Distance : 11.1 km")
print("Stops      : 4\n")

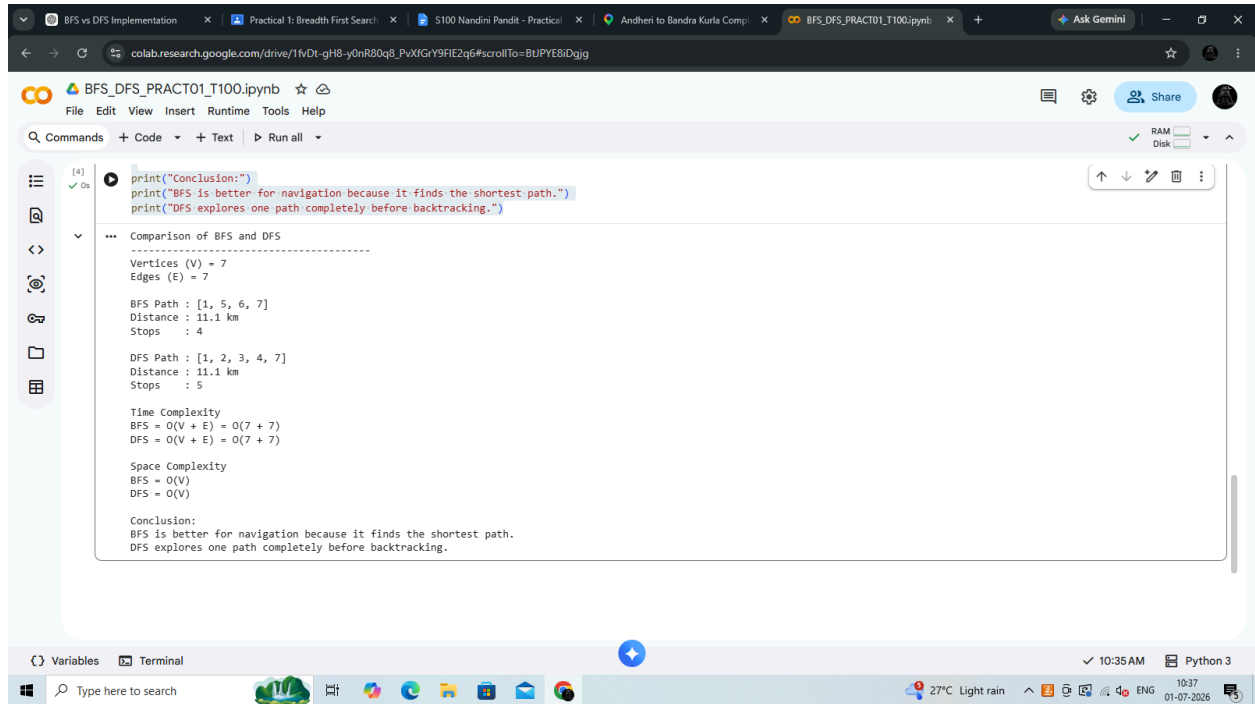
print("DFS Path : [1, 2, 3, 4, 7]")
print("Distance : 11.1 km")
print("Stops      : 5\n")

print("Time Complexity")
print("BFS = O(V + E) = O(7 + 7)")
print("DFS = O(V + E) = O(7 + 7)\n")

print("Space Complexity")
print("BFS = O(V) ")
print("DFS = O(V)\n")

print("Conclusion:")
print("BFS is better for navigation because it finds the shortest path.")
print("DFS explores one path completely before backtracking.")
```

MVLU COLLEGE.



```
[4]: print("Conclusion:")
print("BFS is better for navigation because it finds the shortest path.")
print("DFS explores one path completely before backtracking.")

... Comparison of BFS and DFS
-----
Vertices (V) = 7
Edges (E) = 7

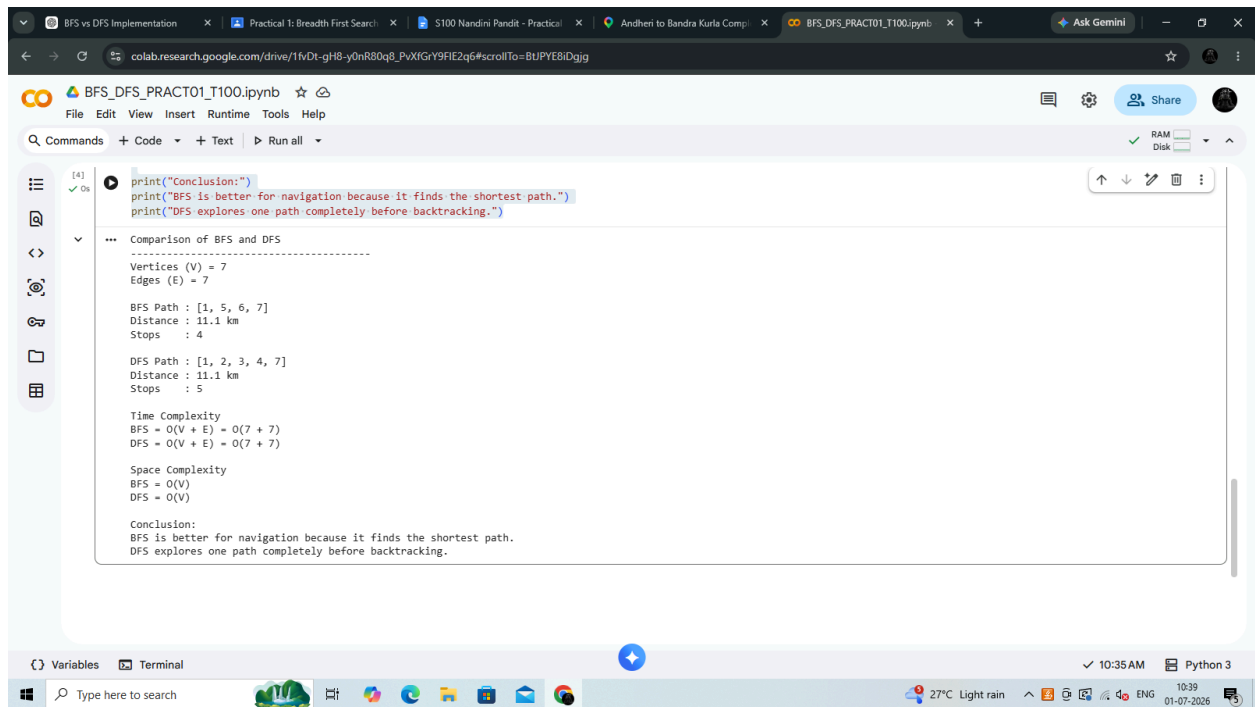
BFS Path : [1, 5, 6, 7]
Distance : 11.1 km
Stops : 4

DFS Path : [1, 2, 3, 4, 7]
Distance : 11.1 km
Stops : 5

Time Complexity
BFS = O(V + E) = O(7 + 7)
DFS = O(V + E) = O(7 + 7)

Space Complexity
BFS = O(V)
DFS = O(V)

Conclusion:
BFS is better for navigation because it finds the shortest path.
DFS explores one path completely before backtracking.
```



```
[4]: print("Conclusion:")
print("BFS is better for navigation because it finds the shortest path.")
print("DFS explores one path completely before backtracking.")

... Comparison of BFS and DFS
-----
Vertices (V) = 7
Edges (E) = 7

BFS Path : [1, 5, 6, 7]
Distance : 11.1 km
Stops : 4

DFS Path : [1, 2, 3, 4, 7]
Distance : 11.1 km
Stops : 5

Time Complexity
BFS = O(V + E) = O(7 + 7)
DFS = O(V + E) = O(7 + 7)

Space Complexity
BFS = O(V)
DFS = O(V)

Conclusion:
BFS is better for navigation because it finds the shortest path.
DFS explores one path completely before backtracking.
```